

LAPORAN PRATIKUM STRUKTUR DATA

STACK

Disusun Oleh:

Endy Pardilian 2511531017

Dosen Pengampu:

Wahyudi. Dr., S.T,M.T

Asisten Pratikum:

M Galid Avero



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2026

KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat-Nya laporan praktikum ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai bentuk pemahaman terhadap materi Struktur Data, khususnya penggunaan Stack dalam bahasa pemrograman Java.

Dalam praktikum ini, penulis mengimplementasikan berbagai operasi pada Stack seperti push, pop, peek, dan pengecekan data. Penerapan Stack dilakukan pada beberapa studi kasus seperti pengelolaan data siswa, evaluasi postfix, pencarian nilai maksimum, serta browser history. Dengan adanya implementasi ini, diharapkan dapat meningkatkan pemahaman dalam mengelola data secara terstruktur menggunakan konsep Stack.

Padang, 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI.....	iii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan.....	1
1.3 Manfaat.....	1
BAB II PEMBAHASAN.....	2
2.1 stakArray_2511531017.....	2
2.2 stackArrayDriver_2511531017.....	4
2.3 NilaiMaksimum_2511531017.....	5
2.4 StackPostfix_2511531017.....	7
2.5 SiswaStack_2511531017.....	9
TUGAS.....	13
2.6 Website_2511531017.....	13
2.7 Browser_2511531017.....	14
BAB III KESIMPULAN.....	19
3.1 Kesimpulan.....	19
3.2 Saran.....	19
DAFTAR PUSTAKA.....	20

BAB I PENDAHULUAN

1.1 Latar Belakang

Perkembangan pemrograman saat ini menuntut penggunaan struktur data yang efisien dalam mengelola data secara terorganisir. Salah satu struktur data yang sering digunakan adalah Stack, yaitu struktur data yang menerapkan konsep Last In First Out (LIFO), di mana data yang terakhir masuk akan menjadi data yang pertama keluar. Konsep ini banyak digunakan dalam berbagai permasalahan seperti pengelolaan riwayat, evaluasi ekspresi, dan manajemen data sementara.

Dalam praktikum ini, Stack diimplementasikan pada beberapa kasus seperti pengelolaan data siswa, perhitungan ekspresi postfix, pencarian nilai maksimum, dan browser history. Melalui penerapan tersebut, diharapkan dapat memberikan pemahaman yang lebih jelas mengenai cara kerja Stack serta penerapannya dalam pemrograman Java untuk menyelesaikan berbagai permasalahan.

1.2 Tujuan

1. Memahami konsep dan cara kerja struktur data Stack dalam Java.
2. Mengimplementasikan operasi dasar Stack seperti push, pop, dan peek.
3. Mampu menerapkan Stack dalam penyelesaian masalah pemrograman.

1.3 Manfaat

1. Menambah pemahaman tentang penggunaan Stack dalam pemrograman.
2. Melatih kemampuan dalam mengelola data menggunakan konsep LIFO.
3. Meningkatkan kemampuan problem solving dengan struktur data Stack.

BAB II PEMBAHASAN

2.1 satckArray_2511531017

```
package pekan3_2511531017;

public class stackArray_2511531017 {
    static final int MAX = 1000;
    int top_1017;
    int a[] = new int[MAX];
    boolean isEmpty(){
        return (top_1017 < 0);
    }
    stackArray_2511531017(){
        top_1017 = -1;
    }
    boolean push (int x){
        if (top_1017>= (MAX - 1)) {
            System.out.println("Stack Overflow");
            return false;
        } else {
            a[++top_1017]= x;
            System.out.println(x + " dimasukkan dalam stack");
            return true;
        }
    }
    int pop() {
        if (top_1017 < 0) {
            System.out.println("Stack Underflow");
            return 0;
        } else {
            int x = a [top_1017--];
            return x;
        }
    }
    int peek (){
        if (top_1017 < 0) {
            System.out.println("Stack Underflow");
            return 0;
        }
        else {
            int x = a[top_1017];
            return x;
        }
    }
    void print () {
        for (int i = top_1017; i>=-1;i--) {
            System.out.print(" " + a[i]);
        }
    }
}
```

Kode 2.1

Penjelasan:

static final int MAX = 1000;

Menentukan kapasitas maksimal stack, yaitu 1000 elemen. Nilai ini bersifat tetap (konstan) karena menggunakan `final`.

```
int top_1017;
```

```
int a[] = new int[MAX];
```

top_1017 digunakan sebagai penanda posisi paling atas stack. Array a digunakan sebagai tempat penyimpanan elemen stack

```
boolean isEmpty()
```

```
{
```

```
    return (top_1017 < 0);
```

Mengecek apakah stack kosong. Jika top_1017 kurang dari 0, berarti tidak ada elemen dalam stack.

```
stackArray_2511531017(){
```

```
    top_1017 = -1;
```

Constructor untuk inisialisasi stack. Nilai top_1017 diset -1 yang menandakan stack masih kosong.

```
boolean push (int x)
```

```
{
```

```
    if (top_1017 >= (MAX - 1)) {
```

```
        System.out.println("Stack Overflow");
```

```
        return false;
```

```
    } else {
```

```
        a[++top_1017] = x;
```

```
        System.out.println(x + " dimasukkan dalam stack");
```

```
        return true;
```

Method untuk menambahkan data ke stack. Jika stack penuh akan muncul pesan “Stack Overflow”, jika tidak maka elemen dimasukkan ke posisi paling atas.

```
int pop() {
```

```
    if (top_1017 < 0) {
```

```
        System.out.println("Stack Underflow");
```

```
        return 0;
```

```
    } else {
```

```
        int x = a[top_1017--];
```

```
        return x;
```

Method untuk menghapus elemen teratas stack. Jika stack kosong akan muncul “Stack Underflow”, jika tidak maka elemen paling atas dihapus dan dikembalikan.

```
int peek ()
```

```
{
```

```
    if (top_1017 < 0) {
```

```
        System.out.println("Stack Underflow");
```

```
        return 0;
```

```
    }
```

```
    else {
```

```
        int x = a[top_1017];
```

```
        return x;
```

Method untuk melihat elemen paling atas tanpa menghapusnya dari stack.

```

void print () {
    for (int i = top_1017; i>-1;i--) {
        System.out.print(" " + a[i]);
    }
}

```

Menampilkan seluruh isi stack dari atas ke bawah menggunakan perulangan.

Class ini adalah implementasi Stack menggunakan array, dengan operasi utama push (tambah), pop (hapus), peek (lihat atas), dan print (tampilkan isi)

2.2 stackArrayDriver_2511531017

```

package pekan3_2511531017;

public class stackArrayDriver_2511531017 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        stackArray_2511531017 s = new stackArray_2511531017();
        s.push(10);
        s.push(20);
        s.push(30);
        System.out.println(s.pop() + " dikeluarkandari stack");
        System.out.println("Elementeratasadalah :" + s.peek());
        System.out.print("Element pada stack:");
        s.print();
    }
}

```

Kode 2.2

Penjelasan:

```
stackArray_2511531017 s = new stackArray_2511531017();
```

Membuat objek stack `s` dari class `stackArray` untuk digunakan dalam operasi stack.

```

s.push(10);
s.push(20);
s.push(30);

```

Menambahkan elemen ke dalam stack secara berurutan. Data 10, 20, dan 30 dimasukkan menggunakan method `push`, sehingga 30 menjadi elemen paling atas.

```
System.out.println(s.pop() + " dikeluarkan dari stack");
```

Menghapus elemen paling atas stack menggunakan `pop()`, lalu menampilkan nilai yang dihapus.

```
System.out.println("Elemen teratas adalah :" + s.peek());
```

Menampilkan elemen paling atas stack tanpa menghapusnya menggunakan method `peek()`.

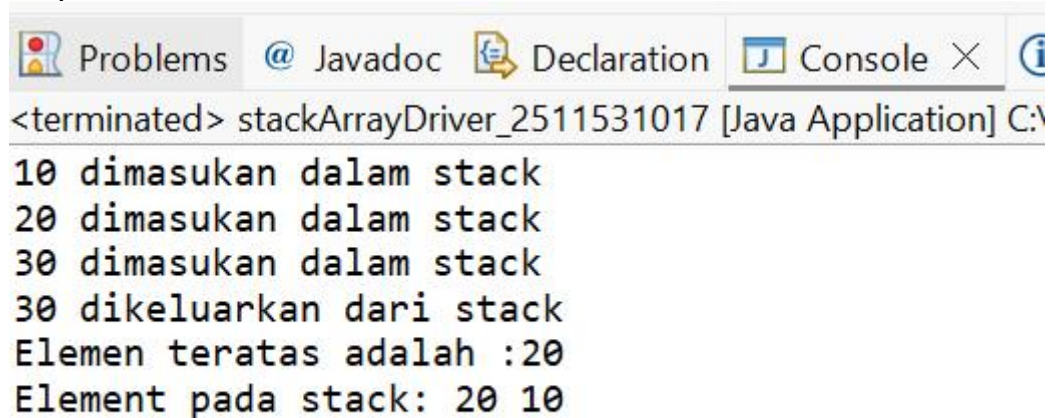
```

System.out.print("Element pada stack:");
s.print();

```

Menampilkan seluruh isi stack dari atas ke bawah menggunakan method `print()`.

Output:



```
<terminated> stackArrayDriver_2511531017 [Java Application] C:\
10 dimasukkan dalam stack
20 dimasukkan dalam stack
30 dimasukkan dalam stack
30 dikeluarkan dari stack
Elemen teratas adalah :20
Element pada stack: 20 10
```

Gambar 2.2

2.3 NilaiMaksimum_2511531017

```
package pekan3_2511531017;
import java.util.*;
public class NilaiMaksimum_2511531017 {
    public static int max(Stack<Integer> s_1017) {
        Stack<Integer> backup_1017 = new Stack<Integer>();
        int maxValue_1017 = s_1017.pop();
        backup_1017.push(maxValue_1017);
        while (!s_1017.isEmpty()) {
            int next = s_1017.pop();
            backup_1017.push(next);
            maxValue_1017 = Math.max(maxValue_1017, next);
        }
        while (!backup_1017.isEmpty()) {
            s_1017.push(backup_1017.pop());
        }
        return maxValue_1017;
    }
    public static void main(String[] args) {
        // TODO Auto-generated method stub
        Stack<Integer> s_1017 = new Stack<Integer>();
        s_1017.push(70);
        s_1017.push(12);
        s_1017.push(20);
        System.out.println("isis stack "+s_1017);
        System.out.println("Stack Teratas " + s_1017.peek());
        System.out.println("Nilai maksimum "+max(s_1017));
    }
}
```

Kode 2.3

Penjelasan:

```
public static int max(Stack<Integer> s_1017)
```

Mencari nilai maksimum dari elemen dalam stack tanpa merusak isi stack secara permanen. Proses dilakukan dengan membandingkan setiap elemen yang diambil dari stack.

```
Stack<Integer> backup_1017 = new Stack<Integer>();  
int maxValue_1017 = s_1017.pop();  
backup_1017.push(maxValue_1017)
```

Membuat stack bantuan untuk menyimpan sementara elemen. Elemen pertama diambil dari stack utama untuk dijadikan nilai awal maksimum. Langkah ini juga memastikan proses perbandingan bisa dimulai.

```
while (!s_1017.isEmpty()) {  
    int next = s_1017.pop();  
    backup_1017.push(next);  
    maxValue_1017 = Math.max(maxValue_1017, next);  
}
```

Melakukan perulangan sampai stack utama kosong, setiap elemen diambil, disimpan ke backup stack, dan dibandingkan untuk mencari nilai terbesar menggunakan `Math.max()`. Proses ini memastikan semua elemen ikut diperiksa.

```
while (!backup_1017.isEmpty()) {  
    s_1017.push(backup_1017.pop());  
}
```

Mengembalikan isi stack dari backup ke stack utama agar kondisi stack tetap seperti semula setelah proses pencarian. Hal ini menjaga data tidak berubah setelah operasi selesai.

```
return maxValue_1017;
```

Mengembalikan nilai terbesar yang ditemukan dalam stack. Nilai ini menjadi hasil akhir dari proses pencarian maksimum.

```
Stack<Integer> s_1017 = new Stack<Integer>();  
s_1017.push(70);  
s_1017.push(12);  
s_1017.push(20);
```

Membuat stack dan memasukkan data 70, 12, dan 20 ke dalamnya menggunakan `push`. Data dimasukkan secara berurutan sehingga membentuk urutan stack.

```
System.out.println("Isi stack " + s_1017);
```

Menampilkan isi stack sebelum proses pencarian maksimum. Ini digunakan untuk melihat kondisi awal data.

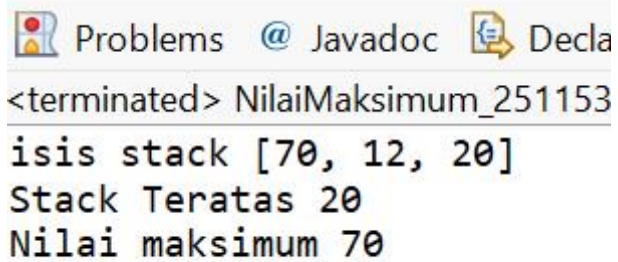
```
System.out.println("Stack Teratas " + s_1017.peek());
```

Menampilkan elemen paling atas stack tanpa menghapusnya. Method `peek()` hanya mengakses data teratas.

```
System.out.println("Nilai maksimum " + max(s_1017));
```

Memanggil method `max()` untuk mencari dan menampilkan nilai terbesar dalam stack. Hasil yang ditampilkan adalah nilai maksimum dari seluruh elemen.

Output:



```
<terminated> NilaiMaksimum_251153
isis stack [70, 12, 20]
Stack Teratas 20
Nilai maksimum 70
```

Gambar 2.3

2.4 StackPostfix_2511531017

```
package pekan3_2511531017;
import java.util.*;
public class StackPostfix_2511531017 {
    public static int postfixEvaluate_1017 (String expression) {
        Stack <Integer> s = new Stack<Integer>();
        Scanner input = new Scanner (expression);
        while (input.hasNext()) {
            if (input.hasNextInt()) {
                s.push(input.nextInt());
            }else {
                String operator_1017 = input.next();
                int operand2 = s.pop();
                int operand1 = s.pop();
                if (operator_1017.equals("+")) {
                    s.push(operand1 + operand2);
                } else if (operator_1017.equals("-")) {
                    s.push(operand1 - operand2);
                } else if (operator_1017.equals("*")) {
                    s.push(operand1 * operand2);
                }else {
                    s.push(operand1/operand2);
                }
            }
        }
        input.close();
        return s.pop();
    }
    public static void main (String[]args) {
        System.out.println("hasil postfix= "+postfixEvaluate_1017("5 4 2
* + 7 -"));
    }
}
```

Kode 2.4

Penjelasan:

```
public static int postfixEvaluate_1017 (String expression)
```

Method ini digunakan untuk mengevaluasi ekspresi postfix dan menghasilkan nilai akhir dalam bentuk integer. Proses dilakukan menggunakan bantuan stack untuk menyimpan operand.

```
Stack <Integer> s = new Stack<Integer>();
```

```
Scanner input = new Scanner (expression);
```

Membuat stack untuk menyimpan angka dan Scanner untuk membaca ekspresi postfix. Scanner digunakan agar input bisa dipisahkan per token.

```
while (input.hasNext())
```

Perulangan ini berjalan selama masih ada token (angka atau operator) dalam ekspresi. Setiap elemen akan diproses satu per satu.

```
if (input.hasNextInt()) {  
    s.push(input.nextInt());
```

Jika token berupa angka, maka langsung dimasukkan ke stack menggunakan `push()`. Angka ini akan digunakan sebagai operand.

```
else {  
    String operator_1017 = input.next();  
    int operand2 = s.pop();  
    int operand1 = s.pop();
```

Jika token adalah operator, maka diambil dua angka teratas dari stack sebagai operand. Urutan pengambilan penting karena mempengaruhi hasil operasi.

```
if (operator_1017.equals("+")) {  
    s.push(operand1 + operand2);  
} else if (operator_1017.equals("-")) {  
    s.push(operand1 - operand2);  
} else if (operator_1017.equals("*")) {  
    s.push(operand1 * operand2);  
} else {  
    s.push(operand1/operand2);
```

Melakukan operasi sesuai operator yang ditemukan. Hasil operasi langsung dimasukkan kembali ke stack untuk diproses lebih lanjut.

```
input.close();
```

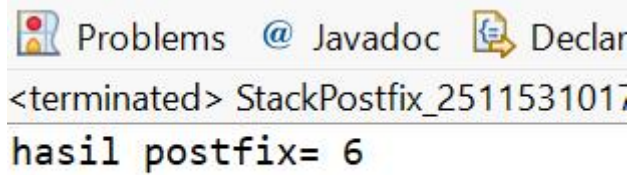
```
return s.pop();
```

Menutup Scanner dan mengembalikan hasil akhir dari stack. Nilai terakhir di stack adalah hasil evaluasi postfix.

```
public static void main (String[] args) {  
    System.out.println("hasil postfix= "+postfixEvaluate_1017("5 4 2  
* + 7 -"));
```

Memanggil method untuk mengevaluasi ekspresi postfix dan menampilkan hasil akhirnya. Ekspresi akan diproses sesuai aturan postfix

Output:



The screenshot shows an IDE interface with tabs for 'Problems', 'Javadoc', and 'Declarations'. The 'Declarations' tab is active, displaying the text: `<terminated> StackPostfix_2511531017` followed by `hasil postfix= 6`.

Gambar 2.4

2.5 SiswaStack_2511531017

```
package pekan3_2511531017;
import java.util.*;
class Siswa_2511531017{
    String nama;
    int nim;

    public Siswa_2511531017(String nama, int nim) {
        this.nama = nama;
        this.nim = nim;
    }

    @Override
    public String toString() {
        return "Nim : " + nim + ", Nama: " + nama;
    }
}
public class SiswaStack_2511531017 {
    private ArrayList<Siswa_2511531017> stack;

    public SiswaStack_2511531017() {
        stack = new ArrayList<>();
    }
    public void push (Siswa_2511531017 mhs) {
        stack.add(mhs);
    }
    public Siswa_2511531017 pop() {
        if (!isEmpty()) {
            return stack.remove(stack.size() - 1);
        }
        return null;
    }
    public Siswa_2511531017 peek() {
        if (!isEmpty()) {
            return stack.get(stack.size() - 1);
        }
        return null;
    }
    public boolean isEmpty() {
```

```

        return stack.isEmpty();
    }

    public void tampilkanSiswa_1017() {
        for (int i = stack.size () - 1; i>= 0; i--) {
            System.out.println(stack.get(i));
        }
    }

    public static void main (String[] args) {
        SiswaStack_2511531017 studentStack = new SiswaStack_2511531017();

        Siswa_2511531017 mhs1 = new Siswa_2511531017 ("Ali",1);
        Siswa_2511531017 mhs2 = new Siswa_2511531017 ("Boby",2);
        Siswa_2511531017 mhs3 = new Siswa_2511531017 ("Charles",3);

        studentStack.push(mhs1);
        studentStack.push(mhs2);
        studentStack.push(mhs3);

        System.out.println("Siswa di dalam stack:");
        studentStack.tampilkanSiswa_1017();

        System.out.println("siswateratas " + studentStack.peek());
        System.out.println("mengeluarkansiswateratasdari stack: " +
studentStack.pop());
        System.out.println("daftar siswasetelah di pop :");
        studentStack.tampilkanSiswa_1017();
    }
}

```

Kode 2.5

Penjelasan:

```

class Siswa_2511531017{
    String nama;
    int nim;

```

Class ini digunakan untuk merepresentasikan data siswa yang terdiri dari nama dan NIM. Data ini menjadi objek yang akan disimpan dalam stack. Class ini berfungsi sebagai blueprint untuk setiap data siswa.

```

public Siswa_2511531017(String nama, int nim) {
    this.nama = nama;
    this.nim = nim;

```

Constructor untuk mengisi nilai atribut nama dan nim saat objek dibuat. Nilai dari parameter disimpan ke dalam atribut class. Hal ini memudahkan proses pembuatan objek siswa.

```

@Override
public String toString() {
    return "Nim : " + nim + ", Nama: " + nama;

```

Method ini digunakan untuk menampilkan data siswa dalam bentuk string. Saat objek dicetak, yang muncul adalah NIM dan nama siswa. Method ini juga mempermudah proses output data.

```

private ArrayList<Siswa_2511531017> stack;

```

Membuat ArrayList yang digunakan sebagai struktur data stack untuk menyimpan objek siswa. ArrayList ini berfungsi sebagai penyimpanan dinamis. Data akan dikelola mengikuti konsep stack (LIFO).

```
public SiswaStack_2511531017() {  
    stack = new ArrayList<>();
```

Constructor untuk menginisialisasi stack. ArrayList dibuat agar siap digunakan untuk operasi stack. Inisialisasi ini wajib dilakukan sebelum data digunakan.

```
public void push (Siswa_2511531017 mhs) {  
    stack.add(mhs);
```

Menambahkan objek siswa ke dalam stack. Elemen selalu ditambahkan di posisi terakhir. Ini sesuai dengan konsep push pada stack.

```
public Siswa_2511531017 pop() {  
    if (!isEmpty()) {  
        return stack.remove(stack.size() - 1);  
    }  
    return null;
```

Menghapus dan mengembalikan elemen paling atas stack. Jika stack kosong, akan mengembalikan null. Method ini mengikuti prinsip LIFO.

```
public Siswa_2511531017 peek() {  
    if (!isEmpty()) {  
        return stack.get(stack.size() - 1);  
    }  
    return null;
```

Melihat elemen paling atas stack tanpa menghapusnya. Method ini hanya mengambil data terakhir. Cocok untuk mengecek isi teratas.

```
public boolean isEmpty() {  
    return stack.isEmpty();
```

Mengecek apakah stack kosong atau tidak. Mengembalikan nilai true jika tidak ada data. Method ini digunakan sebelum operasi pop atau peek.

```
public void tampilkanSiswa_1017() {  
    for (int i = stack.size () - 1; i >= 0; i--) {  
        System.out.println(stack.get(i));
```

Menampilkan seluruh isi stack dari atas ke bawah. Perulangan dilakukan mundur agar sesuai konsep stack (LIFO). Data ditampilkan menggunakan method toString.

```
SiswaStack_2511531017 studentStack = new SiswaStack_2511531017();
```

Membuat objek stack untuk menyimpan data siswa. Objek ini digunakan untuk melakukan operasi push, pop, dan peek. Ini adalah wadah utama data.

```
Siswa_2511531017 mhs1 = new Siswa_2511531017 ("Ali",1);  
Siswa_2511531017 mhs2 = new Siswa_2511531017 ("Boby",2);  
Siswa_2511531017 mhs3 = new Siswa_2511531017 ("Charles",3);
```

Membuat objek siswa dengan data nama dan NIM masing-masing. Objek ini akan dimasukkan ke dalam stack. Setiap objek merepresentasikan satu data siswa.

```
studentStack.push(mhs1);
```

```
studentStack.push(mhs2);
studentStack.push(mhs3);
```

Menambahkan data siswa ke dalam stack secara berurutan. Data terakhir yang dimasukkan menjadi posisi paling atas. Ini menunjukkan konsep LIFO.

```
System.out.println("Siswa di dalam stack:");
studentStack.tampilkanSiswa_1017();
```

Menampilkan seluruh data siswa yang ada di dalam stack. Data ditampilkan dari atas ke bawah. Ini menunjukkan isi stack saat ini.

```
System.out.println("siswa teratas " + studentStack.peek());
```

Menampilkan siswa yang berada di posisi paling atas stack tanpa menghapusnya

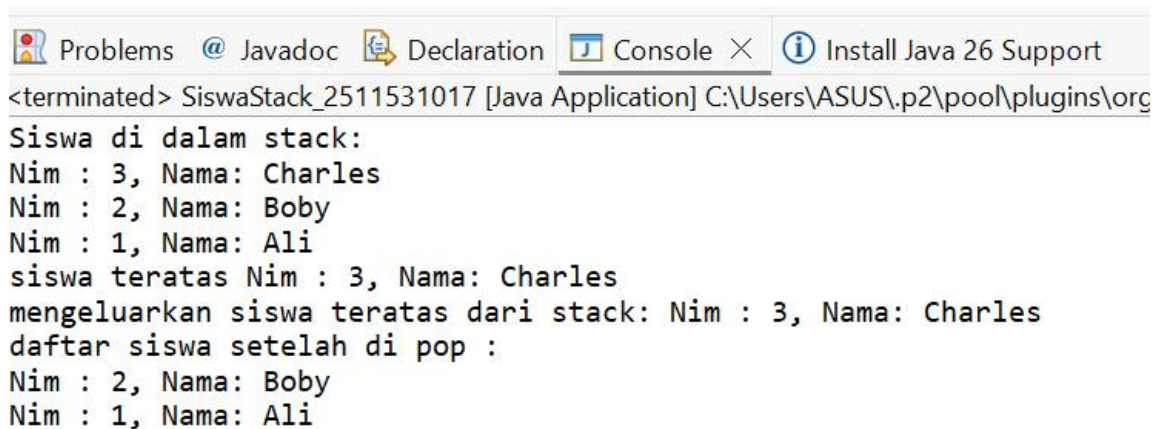
```
System.out.println("mengeluarkan siswa teratas dari stack: " +
studentStack.pop());
```

Menghapus siswa paling atas dari stack dan menampilkannya. Method `pop()` mengembalikan data yang dihapus. Operasi ini mengubah isi stack.

```
System.out.println("daftar siswa setelah di pop :");
studentStack.tampilkanSiswa_1017();
```

Menampilkan isi stack setelah satu data dihapus. Ini menunjukkan perubahan isi stack setelah operasi pop. Data yang tersisa tetap terurut sesuai LIFO.

Output:



```
<terminated> SiswaStack_2511531017 [Java Application] C:\Users\ASUS\.p2\pool\plugins\org
Siswa di dalam stack:
Nim : 3, Nama: Charles
Nim : 2, Nama: Bobby
Nim : 1, Nama: Ali
siswa teratas Nim : 3, Nama: Charles
mengeluarkan siswa teratas dari stack: Nim : 3, Nama: Charles
daftar siswa setelah di pop :
Nim : 2, Nama: Bobby
Nim : 1, Nama: Ali
```

Gambar 2.5

TUGAS

2.6 Website_2511531017

```
package pekan3_2511531017;

public class Website_2511531017 {
    private String judul_1017;
    private String url_1017;

    public Website_2511531017(String judul, String url) {
        this.judul_1017 = judul;
        this.url_1017 = url;
    }
    public String getJudul_1017() {
        return judul_1017;
    }
    public String getUrl_1017() {
        return url_1017;
    }
    public void setJudul_1017(String judul) {
        this.judul_1017 = judul;
    }
    public void setUrl_1017(String url) {
        this.url_1017 = url;
    }
}
```

Kode 2.6

Penjelasan:

```
private String judul_1017;
private String url_1017;
```

Deklarasi atribut untuk menyimpan judul dan URL website. Data ini bersifat private agar hanya bisa diakses melalui method dalam class. Ini menerapkan konsep enkapsulasi.

```
public Website_2511531017(String judul, String url) {
    this.judul_1017 = judul;
    this.url_1017 = url;
}
```

Constructor untuk menginisialisasi objek website. Nilai judul dan URL yang dimasukkan akan disimpan ke atribut class. Constructor ini dipanggil saat objek dibuat.

```
public String getJudul_1017() {
    return judul_1017;
}
```

Getter untuk mengambil nilai judul website. Method ini digunakan untuk mengakses data private dari luar class. Hasil yang dikembalikan berupa string judul.

```
public String getUrl_1017() {
    return url_1017;
}
```

Getter untuk mengambil nilai URL website. Method ini memungkinkan data URL diakses secara aman dari luar class. Nilai yang dikembalikan berupa alamat website.


```
public void setJudul_1017(String judul) {
    this.judul_1017 = judul;
}
```

Setter untuk mengubah nilai judul website. Method ini digunakan jika data judul perlu diperbarui. Perubahan dilakukan melalui parameter input.

```
public void setUrl_1017(String url) {
    this.url_1017 = url;
}
```

Setter untuk mengubah nilai URL website. Method ini memungkinkan pembaruan alamat website. Data lama akan diganti dengan nilai baru, class ini digunakan untuk merepresentasikan data website dengan konsep enkapsulasi (getter dan setter) agar data lebih aman dan terkontrol.

2.7 Browser_2511531017

```
import java.util.Scanner;
import java.util.Stack;
```

```
public class Browser_2511531017 {
```

```
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        Stack<Website_2511531017> history_1017 = new Stack<>();
```

```
        int pilihan_1017;
```

```
        do {
            System.out.println("\n=== Browser History NIM: 2511531017 ===");
            System.out.println("1. Kunjungi Website (Push)");
            System.out.println("2. Tombol Back (Pop)");
            System.out.println("3. Lihat Halaman Aktif (Peek)");
            System.out.println("4. Keluar");
            System.out.print("Pilihan: ");
            pilihan_1017 = input.nextInt();
            input.nextLine();
```

```
            switch (pilihan_1017) {
```

```
                case 1:
```

```
                    System.out.print("Masukkan Judul: ");
                    String judul_1017 = input.nextLine();
```

```
                    System.out.print("Masukkan URL: ");
                    String url_1017 = input.nextLine();
```

```
                    Website_2511531017 web = new
Website_2511531017(judul_1017, url_1017);
                    history_1017.push(web);
```

```
                    System.out.println("Berhasil mengunjungi halaman!");
                    break;
```

```
                case 2:
```

```
                    if (!history_1017.isEmpty()) {
```

```

        Website_2511531017 hapus = history_1017.pop();
        System.out.println("Kembali dari: " +
hapus.getJudul_1017());
    } else {
        System.out.println("History kosong!");
    }
    break;

    case 3:
        if (!history_1017.isEmpty()) {
            Website_2511531017 atas = history_1017.peek();
            System.out.println("Halaman Aktif:");
            System.out.println("Judul: " + atas.getJudul_1017());
            System.out.println("URL: " + atas.getUrl_1017());
        } else {
            System.out.println("Tidak ada halaman aktif!");
        }
        break;

    case 4:
        System.out.println("Keluar dari website");
        break;

    default:
        System.out.println("Pilihan tidak valid!");
}

} while (pilihan_1017 != 4);

input.close();
}
}

```

Kode 2.7

Penjelasan:

Scanner input = new Scanner(System.in);

Stack<Website_2511531017> history_1017 = new Stack<>();

Membuat Scanner untuk input dari user dan Stack untuk menyimpan riwayat website. Stack digunakan karena mengikuti konsep LIFO (Last In First Out). Data terakhir yang masuk akan menjadi halaman aktif.

int pilihan_1017;

Variabel untuk menyimpan pilihan menu dari user. Nilai ini digunakan untuk menentukan proses yang dijalankan

```

do {
    System.out.println("\n== Browser History NIM: 2511531017 ==");
    System.out.println("1. Kunjungi Website (Push)");
    System.out.println("2. Tombol Back (Pop)");
    System.out.println("3. Lihat Halaman Aktif (Peek)");
    System.out.println("4. Keluar");
    System.out.print("Pilihan: ");
}

```

Perulangan do-while digunakan agar menu selalu tampil minimal satu kali. Program akan terus berjalan sampai user memilih keluar. Menampilkan menu pilihan browser

history kepada user. Setiap opsi mewakili operasi stack seperti push, pop, dan peek. Tampilan dibuat agar mudah dipahami.

```
pilihan_1017 = input.nextInt();
input.nextLine();
```

Mengambil input pilihan dari user. `nextLine()` digunakan untuk membersihkan buffer input agar tidak terjadi error.

```
switch (pilihan_1017)
```

Percabangan switch digunakan untuk menentukan aksi berdasarkan pilihan user. Setiap case mewakili fitur yang berbeda.

```
case 1:
```

```
    System.out.print("Masukkan Judul: ");
    String judul_1017 = input.nextLine();
    System.out.print("Masukkan URL: ");
    String url_1017 = input.nextLine();
```

```
    Website_2511531017 web = new Website_2511531017(judul_1017, url_1017);
    history_1017.push(web);
```

```
    System.out.println("Berhasil mengunjungi halaman!");
    break;
```

Case 1 digunakan untuk menambahkan website baru ke dalam stack. Data judul dan URL dimasukkan oleh user, lalu disimpan sebagai objek dan dimasukkan ke history menggunakan push. Ini merepresentasikan fitur kunjungi website.

```
case 2:
```

```
    if (!history_1017.isEmpty()) {
        Website_2511531017 hapus = history_1017.pop();
        System.out.println("Kembali dari: " + hapus.getJudul_1017());
    } else {
        System.out.println("History kosong!");
    }
    break;
```

Case 2 digunakan untuk kembali ke halaman sebelumnya dengan menghapus data paling atas stack. Jika stack tidak kosong, elemen di-pop dan ditampilkan sebagai halaman yang ditinggalkan. Jika kosong, akan muncul pesan bahwa history kosong.

```
case 3:
```

```
    if (!history_1017.isEmpty()) {
        Website_2511531017 atas = history_1017.peek();
        System.out.println("Halaman Aktif:");
        System.out.println("Judul: " + atas.getJudul_1017());
        System.out.println("URL: " + atas.getUrl_1017());
    } else {
        System.out.println("Tidak ada halaman aktif!");
    }
    break;
```

Case 3 digunakan untuk melihat halaman aktif tanpa menghapus data dari stack. Method peek mengambil elemen paling atas untuk ditampilkan. Jika kosong, program memberi informasi bahwa tidak ada halaman aktif.

```

case 4:
    System.out.println("Keluar dari website");
    break;

```

Case 4 digunakan untuk keluar dari program. Program akan berhenti karena kondisi perulangan sudah tidak terpenuhi.

```

default:
    System.out.println("Pilihan tidak valid!");

```

Default digunakan jika user memasukkan angka di luar pilihan menu. Program akan menampilkan pesan bahwa input tidak valid.

```

while (pilihan_1017 != 4);
input.close();

```

Perulangan akan terus berjalan sampai user memilih menu keluar (4). Ini menjaga program tetap aktif selama belum dihentikan.

Oputput:

```

=== Browser History NIM: 2511531017 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar
Pilihan: 1
Masukkan Judul: SSO UNAND
Masukkan URL: https://sso.unand.ac.id/login
Berhasil mengunjungi halaman!

=== Browser History NIM: 2511531017 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar
Pilihan: 1
Masukkan Judul: GitHub
Masukkan URL: github.com
Berhasil mengunjungi halaman!

=== Browser History NIM: 2511531017 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar
Pilihan: 3
Halaman Aktif:
Judul: GitHub
URL: github.com

=== Browser History NIM: 2511531017 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar
Pilihan: 2
Kembali dari: GitHub

=== Browser History NIM: 2511531017 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar
Pilihan: 3
Halaman Aktif:
Judul: SSO UNAND
URL: https://sso.unand.ac.id/login

=== Browser History NIM: 2511531017 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar
Pilihan: 4
Keluar dari website

```

```
=== Browser History NIM: 2511531017 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar
Pilihan: 5
Pilihan tidak valid!

=== Browser History NIM: 2511531017 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar
Pilihan: 2
History kosong!

=== Browser History NIM: 2511531017 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar
Pilihan: 3
Tidak ada halaman aktif!
```

Gambar 2.7

BAB III KESIMPULAN

3.1 Kesimpulan

Stack merupakan struktur data yang bekerja dengan konsep Last In First Out (LIFO), di mana data yang terakhir dimasukkan akan menjadi data yang pertama dikeluarkan. Operasi utama seperti push, pop, dan peek digunakan untuk mengelola data di dalam Stack dengan cara yang terstruktur dan efisien.

Penerapan Stack pada berbagai kasus seperti data siswa, postfix, nilai maksimum, dan browser history menunjukkan bahwa Stack sangat berguna dalam menyelesaikan permasalahan yang membutuhkan pengelolaan data sementara. Dengan memahami konsep ini, kemampuan dalam mengimplementasikan struktur data dalam pemrograman menjadi lebih baik.

3.2 Saran

Sebagai saran, sebaiknya penjelasan materi saat praktikum bisa lebih detail dan perlahan, supaya mahasiswa yang belum terlalu paham coding bisa mengikuti dengan baik.

DAFTAR PUSTAKA

- [1] Oracle, “Class Stack (Java Platform SE),” Oracle Documentation. [Online]. Available: <https://docs.oracle.com/javase/8/docs/api/java/util/Stack.html>
- [2] GeeksforGeeks, “Stack Data Structure in Java,” GeeksforGeeks, 2024. [Online]. Available: <https://www.geeksforgeeks.org/stack-data-structure-in-java/>